Engineering portfolio

Artem Serdiuk | Full Stack Software Engineer

Ukraine | Open to Prague / Czechia relocation and remote opportunities
artom.serdyuk.600@gmail.com | Telegram @Realel
LinkedIn | GitHub | Upwork

What I build

I independently build software that connects business systems and turns complex operational work into explicit, maintainable workflows. I handle requirements, architecture, backend and frontend implementation, integrations, automated checks and deployment tooling.

My primary stack is TypeScript/JavaScript, Node.js and React. I use platform tools where they fit the process and implement custom code for identity, state, validation, recovery and user interaction.

Legal workflow platform

An admin workspace and worker system for email/PDF intake, structured case drafts, document review and external preparation. It includes Microsoft Graph/OneDrive integration and an Office.js Excel client for generating editable PDF forms.

Bidirectional business integration

A PatientNow-GoHighLevel backend covering contacts, appointments, provider calendars, blocked time and purchase history. The current architecture combines ordered event processing, durable receipts, identity history and periodic reconciliation.

MCP and AI workflow platform

An operator application for authenticated service connections, versioned workflow execution and evidence-linked AI drafts. Its engineering focus is reproducible preparation, transaction boundaries and controlled external actions.

How I approach reliability

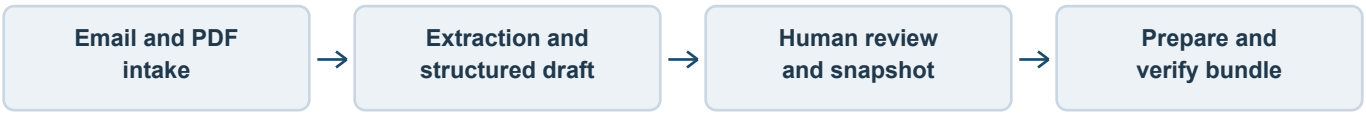
- Keep identity, source versions and human decisions explicit.
- Model partial failures and uncertain outcomes before deciding what is safe to repeat.
- Separate intake, background work, review and externally visible effects.
- Verify persisted or remote results against the reviewed input.

Web application breadth

My frontend and product work includes React/TypeScript CRM interfaces, a Supabase wishlist application, an earlier commercial React/Next.js frontend, and Shopify/GoHighLevel customization.

Legal workflow platform

Independent full stack development



Office.js selected row -> validated values -> isolated editable PDF generation

Problem and ownership

I developed a legal-operations application that turns incoming emails and PDF packages into structured, reviewable work. I implemented the admin workspace, persistence, integrations, background processing, browser preparation, Excel client and validation tooling.

A correction must survive background work

An extraction can finish after an operator changes a field. I implemented field-level edit protection so late results merge into the latest saved draft without overwriting a correction or deliberately blank value. Replacing or removing a PDF invalidates extraction work already in flight.

A reviewed package needs a stable identity

Forms use immutable content-addressed storage. Drafts retain their selected form and rule versions. Approval and queuing happen transactionally, reject stale requests and create a snapshot of case data and documents. Source fingerprints bind the files used by the browser worker to that snapshot.

Remote uncertainty needs its own state

The preparation runner checks exact document types and reopens the saved remote bundle. If a save may have created a partial result, the job enters reconciliation rather than being repeated blindly. The current scope ends at a verified unsubmitted bundle; payment and final court submission are separate actions.

Native Excel and interactive PDFs

The Office.js add-in waits for native recalculation and rechecks the selected row and worksheet. An isolated Python process validates allowed fields, preserves editable PDF controls and checks field/widget/appearance consistency, checkbox state and text fit. Optional OneDrive saves use conflict-safe creation and byte verification.

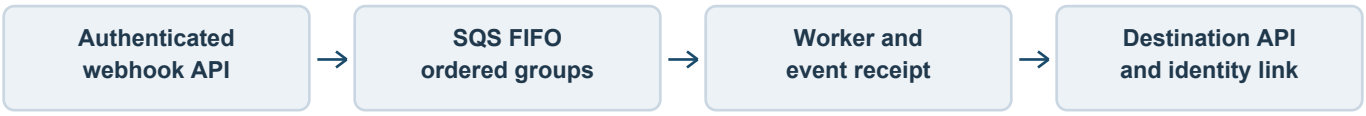
Technology: TypeScript, Node.js, custom HTML/CSS/JavaScript admin UI, SQLite/WAL, Microsoft Graph, Azure Identity, Playwright, Office.js, PDF.js, Tesseract, Python, pypdf, reportlab and Docker.

Validation: local regression checks cover manual edits, stale requests, replacement, restart recovery, auth, Excel selection changes and interactive PDF generation. Desktop Excel acceptance is documented separately from customer-tenant installation.

Source and project documentation

Bidirectional platform integration

PatientNow and GoHighLevel



Periodic reconciliation uses the same domain coordination as event processing.

Problem and ownership

I independently designed and implemented a TypeScript backend connecting two business platforms. The integration covers contacts, appointments, provider calendars, blocked time and purchase history, with changes originating on either side.

Different IDs and delayed events

I implemented persisted cross-platform links, custom-ID lookup and bounded fallback matching. Ambiguous matches and conflicting identities are surfaced instead of choosing the first record. Source event timestamps and authoritative rereads prevent older or eventually consistent data from silently replacing a newer change.

Ordering and recoverable processing

Tenant-aware envelopes enter SQS FIFO groups. The worker renews visibility while processing, stops later work in a failed group and deletes a message only after success. DynamoDB receipts use conditional ownership and expiring processing leases to distinguish completed duplicates from retryable or abandoned work.

Business semantics beyond field copying

Appointment synchronization handles outbound echoes, cancellation tombstones, replacement appointments and timezone/DST cases. Blocked-time fingerprints identify concurrent edits. Purchase mapping preserves the meaning of voids, full refunds, partial refunds and net values.

Deployment tradeoff

I authored Terraform definitions for separate API and worker services, queues/DLQ, receipts, persistent state, TLS, logging and access policies. The topology deliberately keeps a single worker coordinating domain changes while webhook API replicas can scale independently.

Technology: TypeScript, Fastify, Axios, Zod, SQS FIFO, DynamoDB, EFS, ECS/Fargate and Terraform.

Validation: isolated regression tests use synthetic data and blocked outbound networking. They exercise ordering, duplicate events, identity, cancellation, timezones and refund semantics. Live provider/AWS behavior is assessed separately.

The publicly shared source is an earlier polling-based snapshot. This case describes the broader current implementation, including the event-processing and infrastructure extensions.
Architecture case study

MCP and AI workflow platform

Private business application



Problem and ownership

I designed and built an operator application that connects authenticated service accounts, organizes business data, runs versioned preparation workflows and produces source-linked AI drafts for review. The application combines a custom operator UI, service integration and PostgreSQL-backed state.

Reliable authenticated connections

I implemented OAuth metadata discovery, dynamic registration, expiring state and PKCE. Concurrent requests share an in-flight token refresh. Tokens are encrypted per connection, writes are serialized and the account route map is replaced only after a complete discovery pass.

Persistent state and concurrent work

PostgreSQL provides read-only consistent snapshots and serialized transactional mutations with revisions. Preparation runs bind a request to canonical inputs and workflow hashes. Version checks prevent long-running results from silently applying to changed source data or manual edits. An unknown COMMIT outcome stops the store instead of triggering an unsafe retry.

Generated text must refer to real evidence

The server creates bounded source snippets with stable identities, exact offsets and allowed purposes. The model selects evidence options; the server resolves their quotations. Post-generation validation checks references and business constraints, including missing evidence and unknown answers, before a draft reaches human review.

Explicit external action states

Review, preparation, provider preview, acknowledgement and verified delivery are separate states. Transaction-level guards protect pending operations from conflicting edits. Rate-limit queues recover after individual failures, and UI ownership checks stop older asynchronous responses from overwriting newer work.

Technology: JavaScript ES modules, Node.js, Express, PostgreSQL, MCP, OAuth/PKCE, authenticated encryption, n8n and a custom HTML/JavaScript operator UI.

Validation: selected offline regression checks cover OAuth, encrypted storage, source-bound output, canonical identity, state guards, throttling and matching. PostgreSQL integration and actual provider delivery require separate environment-specific checks.

Source is private. A sanitized architecture walkthrough can demonstrate the design and failure handling without exposing client materials.

React applications and frontend work

Product interfaces and service integration

CRM application

I built a React/TypeScript CRM with an Express/MongoDB backend. It combines authentication, user-scoped client and deal management, a sales pipeline, notifications and email workflows. The frontend includes protected routes, asynchronous data flows and error handling.

Engineering focus: bind client/deal queries to the authenticated user, validate request data and represent loading, success and error states consistently across the interface.
Technology: React, TypeScript, Redux Toolkit, Express, MongoDB/Mongoose, JWT, bcrypt, Slack and Nodemailer.
Canonical CRM repository

Wishlist application

I developed a social wishlist interface with authentication, groups, gift reservations, contribution tracking and realtime group chat. Supabase provides the backend services, including PostgreSQL and realtime subscriptions.

Engineering focus: coordinate group interactions, persist reservation/contribution records and update chat views from realtime events. Contribution tracking represents recorded amounts; it is not a payment-transfer gateway.
Technology: React, TypeScript, Tailwind, Supabase and PostgreSQL.
Wishlist repository

Earlier commercial React and Next.js frontend

I implemented frontend functionality for a waiver-management product with user, lawyer and admin dashboards. The source contains document creation/review/signature interfaces, forms, booking-related views and payment-related screens. The original project closed, and the code remains available for technical discussion.

Technology: React, Next.js, Redux, Tailwind, SCSS and Axios.
Frontend repository

Shopify and GoHighLevel

I also build and customize websites, funnels and business workflows. Tasks include cart and product-variant behavior, CRM forms, lead routing, assigned-agent notifications and API/webhook connections. This work combines platform configuration with custom JavaScript and CSS.

Technical discussion

For each project I can explain the business requirement, architecture, my implementation choices, a difficult failure, the tradeoff I made and the checks used to evaluate the result. My main interests are full stack development, integrations and reliable business software.

artom.serdyuk.600@gmail.com | LinkedIn | GitHub